

CSCI-304-03 Computer Organization: Assignment 2

September 21, 2023

Due: 11:55pm 10/5/2023, Thursday

Submission: submit program source file (.c file) on blackboard

For this project, you will write a program in C that displays values in various base representations based on the current value of the program's accumulator. The program's execution is based on the following menu:

```
*****
* Accumulator:          Input Mode: Dec*
* Binary : 0000 0000 0000 0000      *
* Hex : 0000                      *
* Octal : 000000                   *
* Decimal : 0                      *
*****
```

Please select one of the following options:

B Binary Mode
O Octal Mode
H Hexidecimal Mode
D Decimal Mode

C Clear Accumulator
S Set Accumulator

Q Quit
Option:

Note that the value of the accumulator in various base representations is always displayed the menu is displayed. The option can be entered in either upper or lower case (note that the option should be received as a string; the character option will

be located at position 0). **If an invalid option is entered, the program should state that an invalid option has been entered and redisplay the menu.** This behavior should be repeated until the Quit option is selected.

The B, O, H, and D modes allow the user to change the base for which values are entered from the user in setting the accumulator. When a valid mode is selected, the program should print the new mode (e.g., “Mode is Binary”).

The C option resets the accumulator to 0. Note that the initial value of the accumulator should be set to 0 when the program begins.

The S option allows the user to enter a value for the accumulator based on the mode. For octal, hexadecimal, and decimal, standard conversion characters can be used in scanf. For the binary value, the program will need to read the bits into a char array and convert them to a signed short. Note also that a similar conversion function will need to be included to convert the short value of the accumulator to a binary string, with spaces around each nibble, as shown.

Your code must be modular and use functions, switch statements, character arrays, printf, and scanf (not getchar). Create the following functions, no more and no less, with the given interfaces to implement the program (15pt for each function):

```
short get_binary_op (char *bin)
// convert binary in short int; return value

void convert_to_binary (short acc, char *bin)
// format acc in binary for output

short get_operand (char mode)
// read in value in mode; return value

void print_acc (short acc)
// print out accumulator values

char print_menu ()
// print menu and get option

int main ()
// main menu loop; execute option or call appropriate function
```

If you need to assign a **string**, use **strcpy()** (must include string.h). To uppercase a string, use **toupper()** (must include ctype.h). All code should be submitted in a single source file named acc.c. Your program can be compiled with the command:

```
$ gcc acc.c -o acc
```

..and run with the command to test with the sample input file on the blackboard:

```
$ ./acc < text.txt
```

or with the command (to save the output):

```
$ ./acc < text.txt > myacctestout.txt
```

Compare results with sample results on the class webpage using the command:

```
$ diff myacctestout.txt testout.txt
```